Lab 4-Alert Triage with Automation

TABLE OF CONTENTS

#	Section
1	Overview & Objectives
2	Environment Setup
3	Alert Simulation — Triggering the File Download Alert
4	Alert Triage Documentation
5	Step-by-Step Triage Workflow
6	Custom Wazuh Detection Rule
7	Automated VirusTotal Hash Check via Python
8	Obtaining & Configuring the API Key
9	Automated Validation Python Script
10	Expected Script Output
11	TheHive & Cortex Architecture
12	Installing TheHive + Cortex
13	Configuring the VirusTotal Analyzer in Cortex
14	Linking TheHive to Cortex
15	Creating Alert 005 as a TheHive Case
16	Triggering Auto-Analysis via Cortex
17	Configure Auto-Notification in TheHive

1. Overview & Objectives

Tools: Wazuh, VirusTotal, TheHive | Date: 31 March 2026

Wazuh is an open-source Security Information and Event Management (SIEM) platform that collects, aggregates, and analyzes log data from endpoints, servers, cloud workloads, and network devices. In this module you will learn how to triage a Suspicious File Download alert raised by Wazuh, document findings in a structured format, and hand off the case to TheHive for further investigation.

Understand the Wazuh alert hierarchy and rule-set framework.

- Navigate the Wazuh Dashboard and locate Alert ID 005.
- Perform initial triage: classify, enrich, and prioritise the alert.
- Document the alert in the standardised SOC triage table.
- Extract IoCs (Indicator of Compromise) for downstream validation.
- Escalate or close the alert with a documented decision.

2. Environment Setup:

Component	Minimum Spec	Recommended
OS	Ubuntu 20.04 LTS	Ubuntu 22.04 LTS
RAM	4 GB	8 GB
CPU	2 vCPU	4 vCPU
Disk	50 GB	100 GB SSD
Wazuh Manager	v4.3+	v4.7+
Wazuh Dashboard	OpenSearch 2.x	OpenSearch 2.x
Python	3.8+	3.10+

2.1 Install Wazuh (All-In-One via Official Script)

Run the following commands on your Ubuntu server. The installer deploys Wazuh Manager, Indexer (OpenSearch),
and Dashboard in a single node.

```
# Step 1 — Download the Wazuh installation assistant
curl -sO https://packages.wazuh.com/4.7/wazuh-install.sh

# Step 2 — Make it executable
chmod +x wazuh-install.sh

# Step 3 — Run all-in-one installation (takes ~10 minutes)
sudo bash wazuh-install.sh -a

# Step 4 — After install, retrieve auto-generated credentials
sudo tar -O -xvf wazuh-install-files.tar wazuh-install-files/wazuh-passwords.txt
```

# Step 5 — Start / verify services
sudo systemctl status wazuh-manager sudo systemctl status wazuh-indexer sudo systemctl status wazuh-dashboard

2.2 Deploy the Wazuh Agent on the Target Host (10.0.2.20)

The agent runs on the monitored endpoint and forwards events to the Wazuh Manager.
On target host 10.0.2.20 — download & install agent
wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.7.5-1_amd64.deb sudo
WAZUH_MANAGER=" WAZUH_AGENT_NAME='endpoint-10.0.2.20' \ dpkg -i ./wazuh-agent_4.7.5-1_amd64.deb
# Enable and start the agent
sudo systemctl daemon-reload sudo systemctl enable wazuh-agent sudo systemctl start wazuh-agent
# Verify agent status
sudo systemctl status wazuh-agent # On manager — confirm agent appears: sudo /var/ossec/bin/manage_agents -1

3 Alert Simulation — Triggering the File Download Alert:

To practise triage on a realistic alert, we first simulate a suspicious file download from the endpoint at 10.0.2.20. Wazuh's FIM (File Integrity Monitoring) and Syscheck modules will detect the change and generate an alert.

3.1 Enable File Integrity Monitoring

Edit the Wazuh agent configuration on 10.0.2.20:
sudo nano /var/ossec/etc/ossec.conf
Add or verify this block inside <syscheck>:
<syscheck> <disabled>no</disabled> <frequency>300</frequency> <!-- Monitor Downloads directory -->
<directories check_all="yes" report_changes="yes" realtime="yes"> /home </directories> <directories check_all="yes" report_changes="yes" realtime="yes"> /tmp </directories>
<alert_new_files>yes</alert_new_files> </syscheck>
sudo systemctl restart wazuh-agent

3.2 Simulate the Suspicious Download

Execute the following on 10.0.2.20 to simulate a suspicious binary drop:

Create a fake malicious file in /tmp
echo 'X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*' \ > /tmp/suspicious_download.exe
Also simulate wget download behavior
cd /tmp && wget -q -O malware_sample.bin \ https://secure.eicar.org/eicar.com.txt 2>/dev/null || \ echo 'fake-payload-data' > malware_sample.bin
# Force Wazuh syscheck scan immediately
sudo /var/ossec/bin/agent_control -r -u 001 # OR trigger via manager: sudo /var/ossec/bin/wazuh-control restart

3.3 Verify Alert in Wazuh Dashboard

Navigate to the Wazuh Dashboard and locate the generated alert:

1. Open browser → https://<MANAGER_IP> → login with admin credentials.
2. Click Security Events in the left sidebar.

3. Set time range to Last 15 minutes.
4. In the search bar type: agent.ip: 10.0.2.20 AND rule.groups: syscheck
5. Locate the alert with Rule ID 554 (New file added) or 550 (Integrity checksum changed).
6. Click the alert row to expand the full JSON payload.

4. Alert Triage Documentation:

Field	Value
Alert ID	005
Alert Title	Suspicious File Download Detected
Detection Time	30-03-2026 09:14:32 UTC
Source IP	10.0.2.20
Destination IP	External (pending enrichment)
Affected Host	endpoint-10.0.2.20
OS	Ubuntu 22.04 LTS
User Account	ubuntu (uid=1000)
File Path	/tmp/suspicious_download.exe
File Hash (SHA256)	44d88612fea8a8f36de82e1278abb02f (EICAR test)
File Size	68 bytes
Wazuh Rule ID	554 / 550
Rule Group	syscheck, ossec
MITRE Tactic	Execution (TA0002) / Initial Access (TA0001)
MITRE Technique	T1105 — Ingress Tool Transfer
Priority	HIGH
Status	OPEN
Assigned Analyst	SOC Tier-1
Initial Action	Isolate host, extract IoC, forward to TheHive

5. Step-by-Step Triage Workflow

STEP 1 — Acknowledge the Alert
In Wazuh Dashboard, right-click Alert ID 005 → Mark as Acknowledged. This flags the alert as under investigation
and prevents duplicate triage.

STEP 2 — Validate the Source IP
Query the SIEM to confirm 10.0.2.20 is a known internal asset:
Run on Wazuh Manager grep '10.0.2.20' /var/ossec/etc/client.keys # Check agent registration sudo /var/ossec/bin/manage_agents -l | grep '10.0.2.20'

STEP 3 — Extract the Raw Alert JSON
Pull the full alert payload for documentation:
Retrieve alert from Wazuh Indexer (OpenSearch REST API) curl -k -u admin:PASSWORD \
'https://localhost:9200/wazuh-alerts-*/_search?q=agent.ip:10.0.2.20' \ -H 'Content-Type: application/json' | python3 -m json.tool

STEP 4 — Hash the Suspicious File
On the endpoint 10.0.2.20, compute the cryptographic hash:
Compute multiple hash formats md5sum /tmp/suspicious_download.exe sha1sum

/tmp/suspicious_download.exe sha256sum /tmp/suspicious_download.exe # Expected output (EICAR test
file): # 44d88612fea8a8f36de82e1278abb02f suspicious_download.exe

STEP 5 — Check Network Connections from 10.0.2.20
Identify any active outbound connections:
On endpoint — check active connections ss -tunap | grep ESTABLISHED netstat -tunap 2>/dev/null
Check recent DNS queries cat /var/log/syslog | grep '10.0.2.20' | grep -i 'dns\|resolv' # Wazuh API
— get network events for this agent curl -k -u admin:PASSWORD \
'https://localhost:55000/syscollector/001/netiface' \ -H 'Authorization: Bearer TOKEN'

STEP 6 — MITRE ATT&CK; Mapping
Map the observed behaviour to MITRE ATT&CK; framework

STEP 7 — Determine Disposition
Based on evidence, choose one of: True Positive → escalate to TheHive | False Positive → document and tune the
rule | Benign Positive → whitelist with justification.

Tactic	Technique ID	Technique Name	Observed Evidence
Initial Access	T1566.002	Spearphishing Link	File appeared via download
Execution	T1105	Ingress Tool Transfer	Binary dropped in /tmp
Persistence	T1547	Boot/Logon Autostart	Investigate startup entries
Defense Evasion	T1036	Masquerading	.exe disguised in /tmp

6. Custom Wazuh Detection Rule

Binary dropped in /tmp
Investigate startup entries
.exe disguised in /tmp
Create a dedicated rule to fire on executable files downloaded to /tmp or /home from IP 10.0.2.20. Place this in
/var/ossec/etc/rules/local_rules.xml:

<!-- Custom rule: Suspicious file download from internal host --> <group name="syscheck,local,">
<rule id="100200" level="12"> <if_sid>554</if_sid> <field
name="file">\.exe$|\.bin$|\.sh$|\.py$</field> <description> Alert 005: Suspicious executable file
added — possible download </description> <group>pci_dss_11.5,gpg13_4.13,gdpr_II_5.1.f</group>
<mitre> <id>T1105</id> </mitre> </rule> <rule id="100201" level="15"> <if_sid>100200</if_sid>
<field name="agent.ip">10\.0\.2\.20</field> <field name="file">^/tmp/|^/home/</field> <description>
HIGH: Suspicious binary in /tmp or /home on 10.0.2.20 </description> </rule> </group>
# Validate rule syntax and reload
sudo /var/ossec/bin/ossec-logtest sudo systemctl reload wazuh-manager

7. Automated VirusTotal Hash Check via Python

7.1 VirusTotal Overview:

VirusTotal is a free online threat intelligence service that aggregates results from over 70 antivirus engines, URL/domain scanners, and file analysis sandboxes. The VirusTotal API v3 allows programmatic querying of file hashes, IPs, domains, and URLs — enabling automated validation of IoCs extracted from Wazuh alerts.

Endpoint	Purpose
GET /files/{hash}	Query file reputation by MD5/SHA1/SHA256
GET /ip_addresses/{ip}	Query IP reputation
GET /domains/{domain}	Query domain reputation
POST /files	Upload file for scanning
GET /analyses/{id}	Retrieve scan analysis results

8. Obtaining & Configuring the API Key

- Visit https://www.virustotal.com and create a free account.
- Navigate to your profile → API Key tab.
- Copy the 64-character hexadecimal API key.
- Store it securely as an environment variable (never hard-code it):

Store API key securely
export VT_API_KEY='YOUR_64_CHAR_API_KEY_HERE' # Make persistent across reboots:
echo 'export
VT_API_KEY=YOUR_KEY' >> ~/.bashrc source ~/.bashrc

9. Automated Validation Python Script:

```
#!/usr/bin/env python3 """ vt_validator.py — Automated IoC Validation via VirusTotal API v3 Alert
ID 005 | Source: 10.0.2.20 | Date: 30-03-2026 """ import os, sys, json, time, hashlib import
requests from datetime import datetime #  Configuration
 API_KEY =
os.environ.get("VT_API_KEY", "") BASE_URL = "https://www.virustotal.com/api/v3" HEADERS =
{"x-apikey": API_KEY, "Accept": "application/json"} ALERT = { "id" : "005", "source_ip" :
"10.0.2.20", "file_path" : "/tmp/suspicious_download.exe", "md5" :
"44d88612fea8a8f36de82e1278abb02f", "sha256" :
"275a021bbfb6489e54d471899f7db9d1663fc695ec2fe2a2c4538aabf651fd0f", "date" : "30-03-2026", }
def
check_file_hash(hash_val: str) -> dict: """Query VirusTotal for a file hash (MD5/SHA1/SHA256)."""
url = f"{BASE_URL}/files/{hash_val}" r = requests.get(url, headers=HEADERS, timeout=30) if
r.status_code == 200: return r.json() elif r.status_code == 404: return {"error": "hash_not_found",
"hash": hash_val} else: r.raise_for_status() def check_ip(ip: str) -> dict: """Query VirusTotal for
an IP address reputation."""  url = f"{BASE_URL}/ip_addresses/{ip}" r = requests.get(url,
headers=HEADERS, timeout=30) if r.status_code == 200: return r.json() else: r.raise_for_status()
def parse_file_result(data: dict) -> dict: """Extract key fields from VT file response."" if
"error" in data: return {"status": "NOT FOUND", "detections": 0, "total": 0} attrs =
data.get("data", {}).get("attributes", {}) stats = attrs.get("last_analysis_stats", {}) return {
"status" : "FOUND", "malicious" : stats.get("malicious", 0), "suspicious" : stats.get("suspicious",
0), "undetected" : stats.get("undetected", 0), "total_engines": sum(stats.values()), "name" :
attrs.get("meaningful_name", "unknown"), "type_tag" : attrs.get("type_tag", "unknown"),
"reputation" : attrs.get("reputation", 0), "verdict" : "MALICIOUS" if stats.get("malicious",0) > 3
else "CLEAN", } def parse_ip_result(data: dict) -> dict: """Extract key fields from VT IP
response."" attrs = data.get("data", {}).get("attributes", {}) stats =

attrs.get("last_analysis_stats", {}) return { "country" : attrs.get("country", "Unknown"), "owner"
: attrs.get("as_owner", "Unknown"), "malicious" : stats.get("malicious", 0), "total_engines":
sum(stats.values()), "reputation" : attrs.get("reputation", 0), "verdict" : "MALICIOUS" if
stats.get("malicious",0) > 2 else "CLEAN", } def generate_report(hash_result, ip_result) -> str:
"""Generate 50-word summary report."" h = parse_file_result(hash_result) i =
parse_ip_result(ip_result) return ( f"Alert 005 | Source: 10.0.2.20 | Date: 30-03-2026. " f"File
hash flagged as {h['verdict']} by {h['malicious']}/{h['total_engines']} " f"VT engines. IP
reputation: {i['verdict']}, " f"{i['malicious']} detections. MITRE T1105. " f"Escalate to TheHive
immediately for case management." ) def main(): if not API_KEY: print("[ERROR] VT_API_KEY

environment variable not set.") sys.exit(1) print(f"\n{'='*60}") print(f" VT Validator — Alert ID:
{ALERT['id']}") print(f" Date: {ALERT['date']} | Source IP: {ALERT['source_ip']}")
print(f"{'='*60}\n") # --- Hash lookup --- print(f"[*] Checking MD5 hash: {ALERT['md5']}")
hash_data = check_file_hash(ALERT['md5']) h = parse_file_result(hash_data) print(f" Verdict :
{h['verdict']}") print(f" Detections : {h['malicious']}/{h['total_engines']} engines") print(f"
File Type : {h.get('type_tag','N/A')}") time.sleep(15) # respect free-tier rate limit # --- IP
lookup --- print(f"\n[*] Checking IP: {ALERT['source_ip']}") ip_data = check_ip(ALERT['source_ip'])
i = parse_ip_result(ip_data) print(f" Country : {i['country']}") print(f" Owner : {i['owner']}")
print(f" Verdict : {i['verdict']}") # --- Summary --- print(f"\n{' '*60}") print("[50-WORD
SUMMARY]") print(generate_report(hash_data, ip_data)) print(f"{' '*60}\n") # --- Save JSON --
output = { "alert_id" : ALERT['id'], "timestamp" : datetime.utcnow().isoformat(), "source_ip" :
ALERT['source_ip'], "hash_md5" : ALERT['md5'], "hash_result" : h, "ip_result" : i, "summary" :
generate_report(hash_data, ip_data), } with open("vt_report_005.json", "w") as f: json.dump(output,
f, indent=2) print("[*] Full report saved to vt_report_005.json") if __name__ == "__main__": main()
# Install dependencies and run
pip3 install requests python3 vt_validator.py

[ Python — Automated VirusTotal Hash Check ]
import requests
```

```
import json
VT_API_KEY = "YOUR_VIRUSTOTAL_API_KEY"
VT_URL = "https://www.virustotal.com/api/v3/files/{hash}"
# File hashes collected from Wazuh alert (suspicious download from 10.0.2.20)
suspicious_hashes = [
    "d41d8cd98f00b204e9800998ecf8427e",   # Empty file — test
    "44d88612fea8a8f36de82e1278abb02f",   # EICAR test virus
    "abc123def456789012345678901234ab",    # Simulated malware hash
]

def check_hash(file_hash):
    headers = {"x-apikey": VT_API_KEY}
    response = requests.get(VT_URL.format(hash=file_hash), headers=headers)
    if response.status_code == 200:
        data = response.json()
        stats = data['data']['attributes']['last_analysis_stats']
        malicious = stats.get('malicious', 0)
        total     = sum(stats.values())
        print(f"[{file_hash[:8]}...] Malicious: {malicious}/{total}")
        return malicious > 5
    elif response.status_code == 404:
        print(f"[{file_hash[:8]}...] Not found in VirusTotal DB")
        return False
    else:
        print(f"Error: {response.status_code}")
        return False
# Run checks
for h in suspicious_hashes:
    is_malicious = check_hash(h)
    if is_malicious:
        print(f"  → ACTION: Block hash {h} in endpoint security policy")
```

10. Expected Script Output

========== VT Validator — Alert ID: 005 Date:
30-03-2026 | Source IP: 10.0.2.20
========== [*]
Checking MD5 hash: 44d88612fea8a8f36de82e1278abb02f Verdict : MALICIOUS Detections : 68/72 engines
File Type : text [*] Checking IP: 10.0.2.20 Country : Private RFC1918 Owner : Internal Network Verdict : CLEAN
[50-WORD SUMMARY] Alert 005 | Source: 10.0.2.20 | Date: 30-03-2026. File hash flagged as MALICIOUS
by 68/72 VT engines. IP reputation: CLEAN, 0 detections. MITRE T1105. Escalate to TheHive immediately for case management.

11. TheHive & Cortex Architecture:

TheHive is an open-source Security Incident Response Platform (SIRP). Cortex is its companion analysis and active-response engine. When TheHive receives a case observable (e.g., a file hash or IP), Cortex automatically dispatches it to configured Analyzers — including the official VirusTotal_GetReport analyzer — and returns structured results back into the case timeline.

Component	Role	Port
TheHive 5	Case/alert management UI & API	9000
Cortex 3	Analyzer & responder orchestrator	9001
Elasticsearch	Data storage backend	9200
VirusTotal API	External threat intel feed	443
Wazuh Manager	SIEM — alert source	55000

12. Installing TheHive + Cortex

12.1 Install Dependencies

sudo apt update && sudo apt install -y openjdk-11-jre-headless \ curl wget gnupg apt-transport-https python3-pip docker.io

12.2 Deploy via Docker Compose (Recommended)

Create /opt/thehive/docker-compose.yml:
version: '3.8' services: elasticsearch: image: docker.elastic.co/elasticsearch/elasticsearch:7.17.12 environment: - discovery.type=single-node - xpack.security.enabled=false - "ES_JAVA_OPTS=-Xms1g -Xmx1g" volumes: - esdata:/usr/share/elasticsearch/data ports: - "9200:9200" thehive: image: strangebee/thehive:5.2 depends_on: - elasticsearch ports: - "9000:9000" environment: - JVM_OPTS=-Xms512m -Xmx1g volumes: - thehive_data:/opt/thp/thehive/data - thehive_index:/opt/thp/thehive/index cortex: image: thehiveproject/cortex:3.1.7 depends_on: - elasticsearch ports: - "9001:9001" volumes: - /var/run/docker.sock:/var/run/docker.sock - cortex_data:/var/db/cortex environment: - JVM_OPTS=-Xms512m -Xmx1g volumes: esdata: thehive_data: thehive_index: cortex_data:
Start the stack
cd /opt/thehive docker-compose up -d # Verify all containers running docker-compose ps

13. Configuring the VirusTotal Analyzer in Cortex

1. Open Cortex UI → http://<SERVER_IP>:9001
2. Login as admin (first-run wizard sets the password).
3. Navigate to Organization → Analyzers tab.
4. Search for VirusTotal_GetReport → click Enable.
5. Click the gear icon → enter your VT API key.
6. Set Rate limiting: 4 req/min (free tier).
7. Click Save → the analyzer turns green (Active).
8. Also enable VirusTotal_Scan for unknown hashes.

14. Linking TheHive to Cortex

In TheHive, go to Admin - Cortex Servers and add:

Setting	Value
Cortex URL	http://cortex:9001
API Key	(from Cortex → API keys → create org key)
Name	Cortex-Primary
Auto-run	Yes — for hash and IP observables

14.1 Create a Cortex API Key (CLI Method)

```
# Generate API key via Cortex REST API
curl -XPOST http://localhost:9001/api/user/cortex/key/renew \ -H 'Content-Type: application/json' \
-u admin:password | jq .key
```

15. Creating Alert 005 as a TheHive Case

Use the TheHive API to programmatically create the case and attach observables:

```
#!/usr/bin/env python3 """thehive_create_case.py — Create Alert 005 case in TheHive""" import
requests, json THEHIVE_URL = "http://localhost:9000" API_KEY = "YOUR_THEHIVE_API_KEY"
HEADERS = {
"Authorization": f"Bearer {API_KEY}", "Content-Type" : "application/json" } # ■■ Step 1: Create the
Case ■■■
```

16. Triggering Auto-Analysis via Cortex:

Once the observable (hash) is added to the case, trigger VirusTotal analysis:
#!/usr/bin/env python3 """cortex_analyze.py — Auto-run VT analyzer on hash observable""" import requests, time, json CORTEX_URL = "http://localhost:9001" API_KEY = "YOUR_CORTEX_API_KEY" HEADERS =
{ "Authorization": f"Bearer {API_KEY}", "Content-Type" : "application/json" } HASH_VALUE = "44d88612fea8a8f36de82e1278abb02f" ANALYZER_ID = "VirusTotal_GetReport_3_1" # from GET /api/analyzer # ■■ Step 1: Submit Job

■■■